

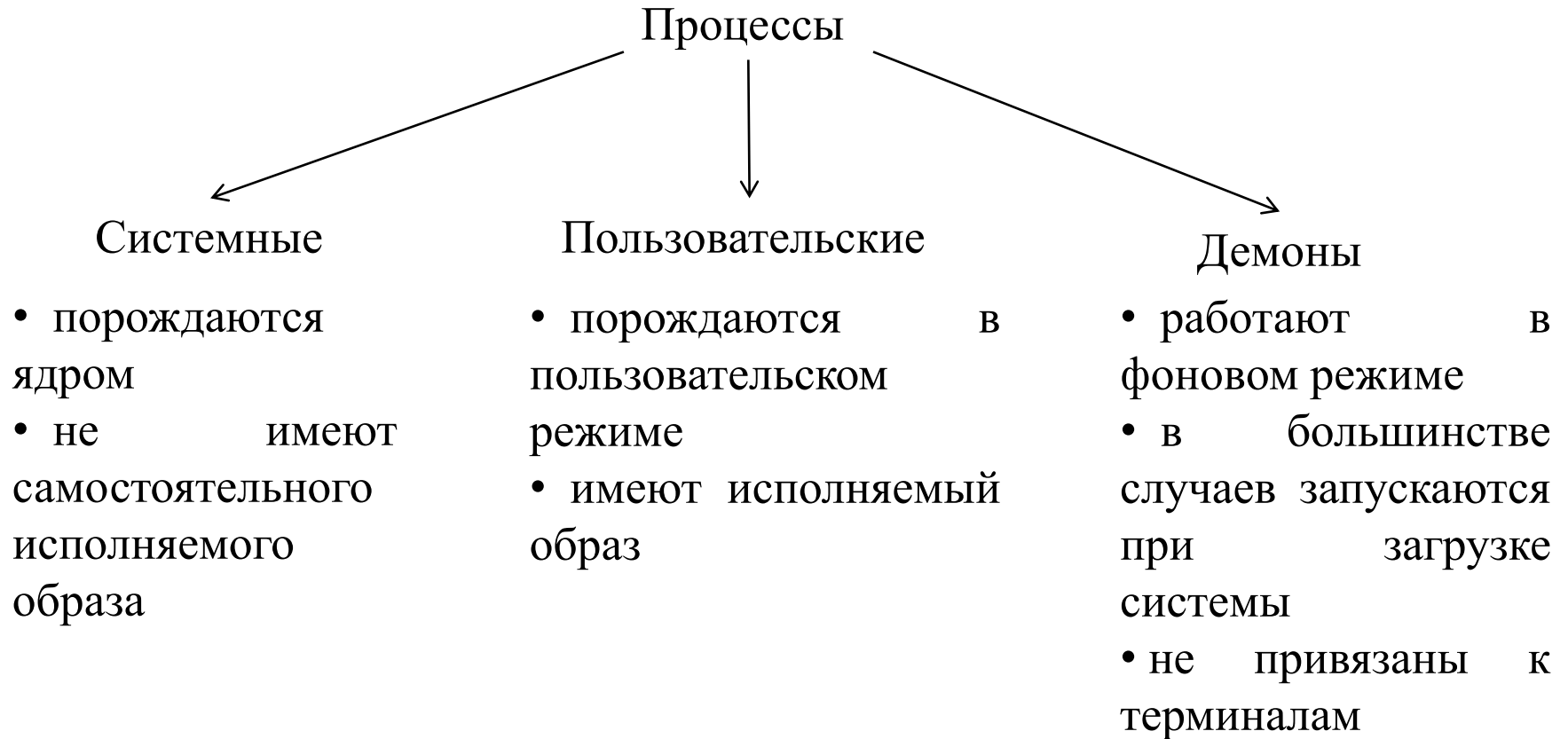
МОДУЛЬ 1

АДМИНИСТРИРОВАНИЕ LINUX

УПРАВЛЕНИЕ ПРОЦЕССАМИ

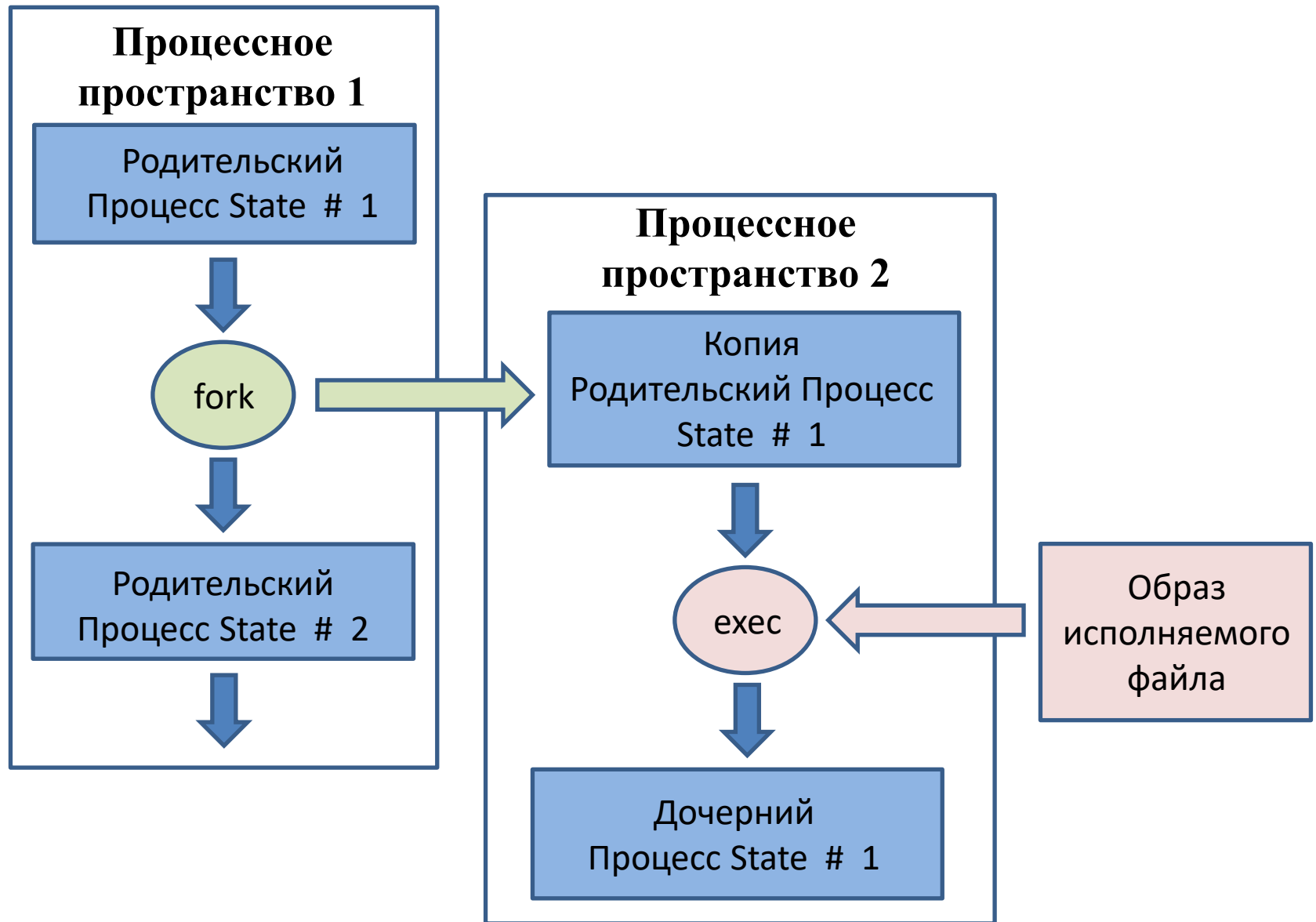
СТРУКТУРА ФС EXT2/3/4

ТИПЫ ПРОЦЕССОВ



Создание процессов в ОС Linux происходит ветвлением с замещением кода родительского процесса

СОЗДАНИЕ ДОЧЕРНЕГО ПРОЦЕССА



КОНТЕКСТЫ ПРОЦЕССА

Контекст процесса — это минимальный набор данных используемый процессом который должен быть сохранён для его прерывания с возможностью последующего выполнения.

➤ **Пользовательский контекст**

(содержимое виртуального адресного пространства, сегментов программного кода, данных, стека, разделяемых сегментов и сегментов файлов, отображаемых в виртуальную память)

➤ **Регистровый контекст**

(содержимое аппаратных регистров: регистр счетчика команд, регистр состояния процессора, регистр указателя стека и регистры общего назначения)

➤ **Системный контекст**

(структуры данных ядра связанные с процессом)

СИСТЕМНЫЕ АТТРИБУТЫ ПРОЦЕССА

1. **PID** (Process ID) – идентификатор процесса
2. **PPID** (Parent PID) – идентификатор родительского процесса
3. **UID** (User ID) – идентификатор пользователя, запустившего процесс
4. **GID** (Group ID) – идентификатор группы запустившего пользователя
5. **EUID** (Effective UID) – идентификатор владельца исполняемого образа; используется для определения прав доступа процесса
6. **EGID** (Effective GID) - – идентификатор группы владельца исполняемого образа
7. **WD** – текущий каталог процесса
8. **TTY** – терминал процесса
9. **NICE** – показатель уступчивости; аналог приоритета процесса

ФАЙЛОВАЯ СИСТЕМА /PROC

Информация о каждом процессе находится в директориях `/proc/<PID>` , где PID – это идентификатор процесса

Системные файлы в директориях `/proc`:

- **cmdline** - список аргументов процесса
- **cwd** - символическая ссылка на текущий рабочий каталог процесса
- **environ** - переменные среды процесса
- **exe** - символическая ссылка на исполняемый файл процесса
- **fd** - подкаталог, содержащий ссылки на файлы, открытые процессом
- **maps** - адресное пространство, выделенное процессу
- **root** - символическая ссылка на корневой каталог процесса
- **mounts** - информация о точках монтирования и типах файловых систем
- **status** - статистическая информация о процессе

КОНВЕЙЕРЫ

Конвейер

- Конвейер перенаправляет стандартный вывод одного процесса в стандартный ввод другого
- Вызывается символом ‘|’

Пример:

```
ls -l | grep MyFolder
```

```
cat /etc/mtab | grep /mnt | wc
```


СИГНАЛЫ

Сигналы - механизм ОС, позволяющий передавать процессам сообщать о некоторых событиях в системе

- Сигналы представляют собой числовые идентификаторы, именованные для удобства пользователя
- В системе зарезервировано 64 сигнала
- Имена сигналов начинаются с префикса “SIG”

Процесс, получив сигнал может:

- игнорировать его
- вызвать системный обработчик
- вызвать собственный обработчик

СИГНАЛЫ

1 SIGHUP	— потеря связи с терминалом
2 SIGINT	— прерывание от клавиатуры. Ctrl-C
6 SIGABRT	— прерывание процесса
9 SIGKILL	— аварийное завершение процесса
10 SIGUSR1	— пользовательский сигнал 1
11 SIGSEGV	— некорректная операция с памятью
12 SIGUSR2	— пользовательский сигнал 2
13 SIGPIPE	— запись в закрытый канал
14 SIGALRM	— сигнал от таймера
17 SIGCHLD	— процесс-потомок завершился
19 SIGSTOP	— остановить процесс
20 SIGTSTP	— приостановить процесс. Ctrl-Z

КОМАНДА PS

Описание: вывести текущее состояние процессов системы

Формат:

ps [Keys]

Ключи:

- e** — вывести все пользовательские процессы
- d** — вывести все пользовательские процессы за исключением служб
- C** COMMAND — вывести процессы, запущенные командой COMMAND
- f** — вывести информацию о процессах в пользовательском виде
- L** — вывести информацию о потоках
- t** TTY1, TTY2, ... — вывести все процессы, привязанные к терминалам
- u** EUSR1, EUSR2, ... — вывести процессы, запущенные с правами и EUSR1, EUSR2

КОМАНДА PS

-U RUSR1, RUSR2, ... – вывести процессы, запущенные пользователями и RUSR1, RUSR2

Пример:

```
#выведет в пользовательском виде информацию о  
# процессах, запущенные командой /bin/bash -u  
ps -f -C /bin/bash -u
```

КОМАНДА PSTREE

Описание: вывести текущее состояние процессов системы в виде дерева

Формат:

```
pstree [Keys] [PID | USER]
```

Ключи:

- a** — показать командную строку процесса
- g** — показывать PGID процесса
- p** — показывать PID процесса
- u** — показывать изменение EUID

Пример:

```
pstree                #выведет все доступные процессы  
pstree -p 1316        #выведет всех потомков процесса 1316
```

КОМАНДА TOP

Описание: отображает информацию о процессах, запущенных в системе, в режиме реального времени

Форма:

```
top [Keys]
```

Ключи:

- c** — вывести все пользовательские процессы в формате пользовательских команд
- d** `ss.tt` — задать частоту обновления (через `ss.tt` секунд)
- n** `NUM` — задать число обновлений
- C** `COMMAND` — вывести процессы, запущенные командой `COMMAND`
- u** `EUID` — вывести информацию о процессах, `euid` которых равны `EUID`
- U** `UID` — вывести информацию о процессах, `uid` которых равны `UID`

КОМАНДА TOP

-p PID1, PID1,..., PID20 – вывести информацию о процессах, PID которых PID1,...,PID20

Пример:

```
#выведет информацию о процессе init (PID=1)
```

```
top -c -p 1
```

КОМАНДА KILL

Описание: посылает сигнал процессу

Формат:

```
kill [-s SIGNAL | -SIGNAL] PID
```

```
kill [-l SIGNAL]
```

Ключи:

-s SIGNAL, **-SIGNAL** — послать процессу PID сигнал SIGNAL

-l — вывести все строковые и числовые идентификаторы сигналов

Пример:

```
#передает процессу с PID=10 сигнал жесткого завершения
```

```
kill -sigkill 10
```

```
kill -s sigkill 10
```


КОМАНДА JOBS

Описание: показать статус задач для текущего сеанса

Формат:

```
jobs [KEYS] [JOB_ID]
```

Ключи:

-p — показывать только PID процессов

-l — вывести подробные данные о задаче в виде:

```
[Job_id] Sym <PID> <Status> <Reason> <Command_line>
```

Пример:

```
#Вывести список задач
```

```
jobs -l
```

```
#показать статус задачи с JOB_ID=1
```

```
jobs -l 1
```

КОМАНДА BG

Описание: запустить задачу в фоновом режиме

Формат:

```
bg [JOB_ID]
```

Вызов без параметра – запуск в фоновом режиме последней задачи

Пример:

```
#приостановить процесс с PID=1316
```

```
kill -SIGTSTP 1316
```

```
#запустить последнюю задачу
```

```
bg
```

```
#запустить задачу с Job_Id=5
```

```
bg 5
```

КОМАНДА FG

Описание: переместить фоновую задачу в синхронный режим

Формат:

```
fg [JOB_ID]
```

Вызов без параметра — запуск в foreground-режиме последней запущенной задачи

Пример:

```
#приостановить процесс с PID=1316
```

```
kill -SIGTSTP 1316
```

```
#запустить последнюю задачу
```

```
bg
```

```
#запустить задачу с Job_Id=5
```

```
fg
```

КОМАНДА NICE

Описание: выполнить команду в отдельном процессе с заданным приоритетом

Формат:

```
nice [KEYS] [COMMAND]
```

Ключи:

-n NUM – присвоить процессу показатель уступчивости на NUM больше, чем по умолчанию

Пример:

```
#запустить bash с NICE=-5  
nice -n 15 /bin/bash
```

КОМАНДА RENICE

Описание: выполнить команду в отдельном процессе с заданным приоритетом

Формат:

```
renice [-n NUM] [-g | -p | -u] ID
```

Ключи:

- n** NUM – присвоить процессу показатель уступчивости на NUM больше
- g** – изменить приоритет всем процессам с GID=ID
- u** – изменить приоритет всем процессам с UID=ID
- p** – изменить приоритет всем процессам с PID=ID

Пример:

```
#изменить NICE у процессе с PID=1316
```

```
renice -n 15 -p 1316
```